



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

#KubeCon #CloudNativeCon
#OpenInfraSummit #PyTorchCon

China 2026

Pathless First: Redefining runc Container Security Against ProcFS-Based Attacks

Lifubang (acmcoder.com, CIIC Group)

- A runc maintainer
- LFAPAC Open Source Evangelist
- Executive Committee Member, ODTC, CCF



runc CVEs: An Overview



China 2026

On November 5, 2025, Cyphar, representing the OCI runc community, released three high-severity vulnerability advisories at one time, all of which are related to writing to the proc filesystem.



Products

Services

Publications

Resources

What's new

[Follow @Openwall on Twitter for new release announcements and other news](#)

[\[<prev\]](#) [\[next>\]](#) [\[thread-next>\]](#) [\[day\]](#) [\[month\]](#) [\[year\]](#) [\[list\]](#)

Message-ID: <2025-11-05-remember-remember-the-fifth-of-november-3EtRdS@cyphar.com>

Date: Wed, 5 Nov 2025 20:53:08 +1100

From: Aleksa Sarai <cyphar@...har.com>

To: oss-security@...ts.openwall.com, fulldisclosure@...lists.org

Subject: runc container breakouts via procfs writes: CVE-2025-31133, CVE-2025-52565, and CVE-2025-52881

| NOTE: This advisory was sent to <security-announce@...ncontainers.org>
| on 2025-10-16. If you ship any Open Container Initiative software, we
| highly recommend that you subscribe to our security-announce list in
| order to receive more timely disclosures of future security issues.
| The procedure for subscribing to security-announce is outlined here:
| <<https://github.com/opencontainers/.github/blob/main/SECURITY.md#disclosure-distribution-list>>

Hello,

This is a notification to vendors that use or ship runc about THREE (3) high-severity vulnerabilities (CVE-2025-31133, CVE-2025-52565, and CVE-2025-52881). All three vulnerabilities ultimately allow (through different methods) for full container breakouts by bypassing runc's restrictions for writing to arbitrary /proc files.

runc CVEs: An Overview



China 2026

Black Wednesday: November 5, 2025

01. CVE-2025-31133

Attackers abuse the ``maskedPaths`` mechanism to replace ``/dev/null``, enabling read-write bind mounting of ``core_pattern``. This grants them the ability to tamper with the host's core dump files, severely compromising the integrity of system auditing and forensics.

02. CVE-2025-52565

By exploiting a vulnerability in the initialization timing window, attackers perform a bind mount of ``/dev/console`` before masking rules take effect. This bypasses security policy restrictions, directly breaking container isolation to achieve escape and gain access to the host machine.

03. CVE-2025-52881

Through ProcFS write redirection, while bypassing both LSM label enforcement mechanisms and sysctl escape protections, this constitutes the most dangerous full-chain container breakout path disclosed this year, allowing direct acquisition of Root-level privileges on the host machine.

Core Question: Is it still safe to deploy AI Agent workloads in runc containers? In the face of such high-severity vulnerabilities, is it necessary to fully migrate all AI agent workloads to container runtimes with stronger isolation capabilities, such as gVisor or Kata Containers?

The Answer: I see it as a matter of preference, similar to choosing between air and rail travel. It ultimately depends on your risk tolerance.

runc CVEs: An Overview



China 2026

Security Practice Insights from a Decade of Operations at acmcode.com: Robust Protection from Massive Code to Intelligent Agents



Running Hundreds of Thousands of Untrusted Code Instances Daily

Hundreds of thousands of untrusted code submissions are executed daily in runc sandboxes. Strict namespace isolation and resource limits ensure malicious code cannot break out of the sandbox, safeguarding the platform's underlying environment.



Handling Tens of Thousands of Untrusted AI Agent Interactions

We handle tens of thousands of daily interaction requests from user-defined AI Agents. Leveraging runc's lightweight virtualization, we achieve strong isolation between Agents, effectively blocking model injection and privilege escalation risks.



A Decade of Zero Escapes and Zero Successful Exploits

Thanks to refined runc configuration hardening, strict adherence to the principle of least privilege, and full-lifecycle security auditing, we have not experienced a single sandbox escape or successful vulnerability exploitation in a decade of high-intensity operations. This proves that proper container security configuration and management are the core defense against attacks.

runc CVEs: An Overview



China 2026



Aleksa Sarai @lordcyphar

"I haven't said this publicly before, but back in April runc was scanned by Mythos and it didn't find anything (apparently it was the first project where this was the case). Funnily enough this means it missed CVE-2026-41579 despite it being similar to previous (very recent) bugs."



1



11



704



Vulnerability Stealth: Blind Spots in Traditional Scanning

Path traversal vulnerabilities like CVE-2026-41579 exploit the complex logic of container runtimes and are highly stealthy. Even professional security scanning tools (e.g., Mythos) failed to detect them in time, revealing the limitations of static detection in complex runtime scenarios.



Security Warning: Single Tools Are Unreliable

This incident warns us that automated security tools cannot fully replace manual audits and dynamic detection. For core container components like runc, we must combine static analysis, fuzzing, and manual code audits to build a stronger security defense.

Core Insight: The security of container infrastructure cannot rely solely on a single tool. A multi-layered defense system of "automated scanning + in-depth manual auditing + continuous runtime monitoring" must be established to ensure zero-trust security for the underlying runtime.

runc CVEs: An Overview



China 2026

The three CVEs of actual concern are:

CVE-2016-9962

MEDIUM SEVERITY

RunC allowed additional container processes spawned via ``runc exec`` to be `ptrace'd` by the container's PID 1, potentially enabling privilege escalation within the container context.

Fixed: Jan 10, 2017

CVSS Score: 6.4

CVE-2019-5736

HIGH SEVERITY

Critical file-descriptor mishandling related to ``/proc/self/exe`` allowed attackers to overwrite the host's runc binary, leading to full host compromise and container escape.

Fixed: Feb 11, 2019

CVSS Score: 7.6

CVE-2024-21626

HIGH SEVERITY

An internal file descriptor leak in runc allowed for container escape by exposing host resources, posing a severe threat to the isolation guarantees of containerized environments.

Fixed: Jan 31, 2024

CVSS Score: 8.6

Thanks to all these Cybersecurity Experts



China 2026

If you have a
correct
configuration,
'runc' is **safe
enough** to run
AI agents
workload.
**It's all
because of
you!**



Aleksa Sarai
SUSE



Li Fubang
ACMcooder · CISC



Akihiro Suda
NTT



Lei Wang
Huawei



Rodrigo Campos Catelin
Microsoft



Tõnis Tiigi
Docker



Alban Crequy
Microsoft



@cclerget



Rory McNamara
Snyk



Dave Trudgian
Sylabs



Arthur Chan
Ada Logistics



Derek Manzella



JUNYI LIU



Etienne Champetier



Felix Wilhelm
Google Project Zero



Kir Kolyshkin
Red Hat



Andrew G. Morgan
Google



Sam Dowell
Google



Daniel Watkins



@olsova

runc CVEs: An Overview



China 2026

All vulnerabilities in the table below are path-related, except for CVE-2016-3697, CVE-2016-9962, CVE-2021-43784, CVE-2022-29162, CVE-2023-25809, and CVE-2023-27561.

CVE 编号	发生了什么	严重度	关键前置条件
CVE-2019-16884	/proc被覆盖挂载	高 7.5	恶意镜像
CVE-2019-19921	procfs 竞态: LSM 标签写入被重定向到假文件	中	恶意镜像 + 共享卷 + 竞态
CVE-2021-30465	符号链接交换: 挂载目标被换到 rootfs 之外	高 7.6	恶意镜像 + 共享卷 + 竞态
CVE-2023-28642	镜像内 /proc 符号链接绕过 AppArmor (rootless)	中	恶意镜像 + 共享卷 + 竞态
CVE-2024-45310	在宿主机创建空文件/目录 (MkdirAll 竞态)	中	恶意镜像 + 共享卷 + 竞态
CVE-2025-31133	maskedPaths 滥用: /dev/null 被调包, core_pattern 以读写被绑定挂载	高 7.3	恶意镜像 + 共享卷 + 竞态
CVE-2025-52565	/dev/console 在屏蔽规则生效前被绑定挂载	高 7.3	恶意镜像 + 共享卷 + 竞态
CVE-2025-52881	procfs 写入重定向: 跳过 LSM 标签 + sysctl 逃逸 gadget	高 7.3	恶意镜像 + 共享卷 + 竞态
CVE-2026-41579	/dev 符号链接欺骗 setupPtmx, 在宿主机上创建快捷方式。	中	恶意镜像 + 共享卷 + 竞态



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

China 2026

Part I

Path-Related Attack Surface Analysis



Proc FS Interface



China 2026



/proc/self/attr/exec

Specifies the security attributes assigned to a process after it invokes the `execve(2)` system call, including SELinux security contexts or AppArmor profiles. It is a critical entry point for process privilege switching.



/proc/self/attr/keycreate

Configures the SELinux security context to be applied to all subsequently created kernel keys, ensuring strict permission isolation and access control for key resources at creation time.



/proc/self/attr/apparmor/exec

An AppArmor-specific interface used to specify the security profile to be loaded after a process executes `execve(2)`. It enforces Mandatory Access Control (MAC) at the application layer, restricting the process's access to system resources.



/proc/sys/net/ipv4/ip_forward

A boolean switch (1=enable, 0=disable) that controls IPv4 packet forwarding. When enabled, the Linux system gains routing capabilities to forward and relay packets between different network interfaces.

Code Example: Set AppArmor RuntimeDefault Policy for a Process via Proc FS

```
err := os.WriteFile("/proc/self/attr/apparmor/exec", []byte("exec RuntimeDefault"), 0o200)
if err != nil { log.Fatalf("Failed to apply AppArmor profile: %v", err) }
```

How to attack?



Risky Code Snippet

The code attempts to set AppArmor policy by writing to a proc filesystem path:

```
os.WriteFile("/proc/self/attr/apparmor/exec", []byte("execRuntimeDefault"), 0)
```

This relies on the assumption that `/proc` is the **real, immutable** kernel interface, which is not guaranteed in a compromised container.



Mount Bypass Attack

Attackers can create a fake `/proc` structure and mount it over the real one:

```
$ mkdir -p  
fakeproc/self/attr/apparmor  
$ touch  
fakeproc/self/attr/apparmor/exec
```

Now writes to `/proc` go to the attacker-controlled directory, not the real kernel interface.



Impact & Mitigation

Severe Consequence: AppArmor security profiles are completely bypassed, allowing arbitrary code execution or container escape.

Effective Fix: Validate the mount point first, check the the mount point to ensure we can't mount on container's `/proc` point..

Key Takeaway: Never implicitly trust the integrity of filesystem paths in containerized environments. Always verify the underlying filesystem and mount points before performing security-critical operations.

So, are we truly safe now?



China 2026



01. Shared Volume Trap

Two containers (A & B) share a volume named "fakeproc", both mounted at the path **"/fakeproc"**.

This shared storage breaks the isolation boundary, creating a hidden channel for cross-container manipulation and potential privilege escalation.



02. Race Condition Bypass

In container B, **"/proc"** is symlinked to **"/fakeproc/evil"**. Container A continuously swaps "evil" and "evil1" files.

This creates a **race window** that confuses the security checkers, allowing attackers to bypass AppArmor profiles and gain unauthorized access.



03. Mitigation Strategy

Implement strict validation during the mount process:

Explicitly **check the mount point** to ensure it is not a symbolic link (symlink). This blocks the malicious redirection at the source, closing the race condition window and enforcing AppArmor policies effectively.

Key Takeaway: Container isolation is not absolute. Shared volumes and symbolic links can create hidden attack vectors. Always validate the integrity of mount points to prevent race condition exploits.

Is it safe enough now?



China 2026



01. Is the Isolation Sufficient?

Container isolation isn't foolproof. A subtle attack vector exists where mounting a malicious directory to `/proc/self/attr` (a subdirectory of `/proc`) can bypass security policies, undermining the container's sandbox protections.



02. The "Fakeproc" Exploit

```
mkdir -p fakeproc/apparmor  
touch fakeproc/apparmor/exec
```

By creating a fake proc structure and mounting it to the sensitive `/proc/self/attr` path inside the container, attackers can forge security attributes.



03. Critical Impact: Bypass AppArmor

This exploit directly bypasses **AppArmor profiles** -- Linux's mandatory access control (MAC) system. It allows attackers to execute arbitrary code with elevated privileges, escaping the container's security boundaries and potentially compromising the host system.



04. Effective Mitigation Strategies

- 1. Mount Point Whitelisting:** Block mounting on any `/proc` child directories unless explicitly allowed in a secure whitelist.
- 2. Superblock Validation:** Check the file descriptor type using `PROC_SUPER_MAGIC` to ensure it references the real proc filesystem, not a fake one.



01. Safety Concern

Is the container environment truly isolated?

The core question lies in whether filesystem mount operations inside a container can be abused to bypass the host's security mechanisms. A seemingly harmless mount could lead to unexpected privilege escalation or resource access.



02. The "Fakeproc" Trap

```
$ mkdir -p fakeproc/apparmor  
$ ln -s /proc/sysrq-trigger \  
fakeproc/apparmor/exec
```

By mounting this crafted directory to `/proc/self/attr``, attackers create a deceptive layer that confuses the kernel's path validation logic.



03. Impact & Fix

Critical Bypass: Evades ``PROC_SUPER_MAGIC`` check and AppArmor profile enforcement.

Severe Risk: Leads to host-level Denial-of-Service (DoS) attacks.

Mitigation: Validate the actual filesystem type of the target path during mount operations.

Key Takeaway: Container isolation is not absolute; always validate mount sources and enforce strict path resolution checks.



China 2026

Part II

Potential Solutions



Root cause



China 2026

```
</>  
Consider `os.WriteFile("/proc/self/attr/apparmor/exec", []byte("exec RuntimeDefault"), 0)`.
```

We fully trust the `/proc` mount point in the container.

But each component of the path could be:



A symlink: can be swapped by the attacker when shared with other containers;



Over-mounted: can be controlled by the attacker when shared with other containers.

Potential solutions



China 2026

⌚ In fact, there is only one solution to fix this issue:
We should not trust any paths in the container.

But there are two potential solutions for zero trust any paths in the container:



If we trust the host's “/proc”, we still face two risks:

1. We can't tolerate mistakes made by administrators;
2. We can't trust “/proc” in nested containers.



Use the direct ``Proc`` file-descriptor handler obtained from the kernel.

This serves as the final solution, even for #1.

new mount API



China 2026



Linux 5.2 introduced a brand-new filesystem mounting API, replacing the traditional mount(2) with a set of file-descriptor-based system calls.



You can create a "detached" mount object without attaching it to any path, and operate on it via file descriptor.



Ref:

<https://github.com/brauner/man-pages-md>

<https://lifubang.github.io/acm/golang-new-mount-api/>

How to get a safe proc fd?



China 2026

1 fsopen

2 fsconfig

fsconfigCreate

4 fsmount, but you don't need to mount it to a real path.

```
99 func newPrivateProcMount(subset bool) (_ *Handle, Err error) {
100     procfsCtx, err := fd.Fsopen("proc", unix.FSOPEN_CLOEXEC)
101     if err != nil {
102         return nil, err
103     }
104     defer procfsCtx.Close() //nolint:errcheck // close failures aren't critical here
105
106     if subset && getProcfsFeatures().hasSubsetPid {
107         // Try to configure hidepid=ptraceable,subset=pid if possible, but
108         // ignore errors.
109         _ = unix.FsconfigSetString(int(procfsCtx.Fd()), "hidepid", "ptraceable")
110         _ = unix.FsconfigSetString(int(procfsCtx.Fd()), "subset", "pid")
111     }
112
113     // Get an actual handle.
114     if err := unix.FsconfigCreate(int(procfsCtx.Fd())); err != nil {
115         return nil, os.NewSyscallError("fsconfig create procfs", err)
116     }
117     // TODO: Output any information from the fscontext log to debug logs.
118     procRoot, err := fd.Fsmount(procfsCtx, unix.FSMOUNT_CLOEXEC, unix.MS_NODEV|unix.MS_NOEXEC|unix.MS_NOSUID)
119     if err != nil {
120         return nil, err
121     }
122     defer func() {
123         if Err != nil {
124             _ = procRoot.Close()
125         }
126     }()
127     return newHandle(procRoot)
128 }
```



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

China 2026

Part III

Pathless Framework Solution



Problems



China 2026

 How to maintain compatibility with Linux kernels older than 5.2?

 How to detect whether the new mount APIs are available in suitable Linux kernels? For example, they may be disabled by seccomp filters.

 How to deal with some old Linux kernels with a backport of the new mount APIs?

 How to cache the ``Proc`` file-descriptor handler?

Compatibility Design



China 2026

✓ Do a ``HasNewMountAPI`` check;

→ Fall back to ``openat2``:

```
</> 1. procRoot, err := os.OpenFile("/proc", unix.O_PATH|unix.O_NOFOLLOW|unix.O_DIRECTORY|unix.O_CLOEXEC,
    0)
    2. Verifies the handle is the root of a procfs filesystem
    3. unix.Openat2(procRoot, unsafePath, &unix.OpenHow{
        Flags:  unix.O_PATH | unix.O_NOFOLLOW | unix.O_CLOEXEC,
        Resolve: unix.RESOLVE_BENEATH | unix.RESOLVE_NO_XDEV | unix.RESOLVE_NO_MAGICLINKS,
    })
```

Compatibility Design



China 2026

→ If ``openat2`` is not available, fall back to the solution:
Traverse each path component one by one, for each one:

 verifyProcHandle



 checkSubpathOvermo
unt



 check whether the
symlink is an absolute
path

For custom-modified legacy kernels



China 2026

- ✗ RHEL 8 has a backport of `fsopen(2)` that appears to have some very difficult-to-debug performance pathology.



cyphar on Aug 1, 2025 Author

...

I've noticed the AlmaLinux 8 tests take too long now (>30m, causing a timeout).

~~I suspect it's because of the excessive `open_tree(2)` calls for procs operations on Linux 4.18 (caused by [cyphar/filepath-securejoin#57](#) which removed the cached procs handle — this was done to avoid [CVE-2024-21626](#) style attacks).~~

~~I was under the impression that `open_tree(OPEN_TREE_CLONE)` was quite efficient (it doesn't cause mount lock contention on new kernels IIRC), but this might be something that was improved in later kernel releases — maybe we need to have a global handle after all...~~

EDIT: Oh, `open_tree` was added in Linux 5.2. The docs in <https://github.com/brauner/man-pages-md/> are wrong. RHEL 8 might have a backport though (which could have performance issues, I guess? 4.18 doesn't have any of the anonymous mount namespace infrastructure IIRC.) -- I would have to check... Maybe the userspace resolver is the issue? Hmm...

Pathless Framework



China 2026



Implementing this framework in production environments is complex.



There is a pure Go implementation of the ``Pathless Framework``:

<https://github.com/cyphar/filepath-securejoin>

Since this is a common issue across all programming languages, as a core maintainer of runc, Sarai (@cyphar) developed the ``libpathrs`` library, which also provides a ``Pathless Framework``:

<https://github.com/cyphar/libpathrs>



You can also use ``libpathrs`` through the ``filepath-securejoin`` interface by setting the ``libpathrs`` build tag.

Known pitfalls



China 2026

✖ Performance issues on some old custom-modified legacy kernels, for example: RHEL 8.

💡 With the Pathless framework, when we access the path `/proc/sys/net/ipv4/ip_unprivileged_port_start``, AppArmor will treat it as `/sys/net/ipv4/ip_unprivileged_port_start``. There is a problematic `/sys`` rule: ``deny /sys/[^fdc]*{,/**} wklx``; we should change it to ``deny /sys/[^fdcn]*{,/**} wklx``.

Mitigations



China 2026

 Use containers with user namespaces (with the host root user not mapped into the container's user namespace); user namespaces have been GA since version `1.36` in Kubernetes.

 Rootless containers, for example: Usernetes (Rootless Kubernetes).

 Do not run untrusted container images.

 Do not bind a sub-path from a shared volume, and do not bind to a sub-path in a shared volume.



China 2026

Part IV Future Work



At the architectural level



China 2026



Nowadays, we only use the new mount API for ``procfs``; we should introduce it to all other types of filesystems, for example: devtmpfs, devpts, or tmpfs.



And we only use it to access paths, not to set up container mounts; we should also apply it in this area. I have done some initial work on this: <https://github.com/opencontainers/runc/pull/5378>



We need more reviewers and contributors in the OCI runc community and its dependencies. For example: opencontainers/runc, cyphar/filepath-securejoin, cyphar/libpathrs

At the vulnerability-research level



China 2026



We should set up a vulnerability-research community to make runc and other container runtimes more secure.



Sometimes someone has ideas about attack surfaces but is too busy to pursue them; other members can help write PoC scripts.



Potential attack surfaces:

- User namespace
- Rootless
- Escape after started

Before your research, please check runc's threat model:
<https://github.com/opencontainers/runc/pull/5361>



LFB
中国大陆



扫二维码，添加我为朋友。



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

China 2026

#KubeCon #CloudNativeCon
#OpenInfraSummit #PyTorchCon

Thanks for listening

Unlock boundless potential, shape a brighter future hand in hand.



Github ID
@lifubang



Website:
www.acmcoder.com



Email:
lifubang@ciic.com.cn